From Region Arrival to Instance-Level Grounding in Vision-and-Language Navigation

Xiangyu Shi[1], **Ruoxi Yang**[2], **Wei Tao**[2], **Jiwen Zhang**[3], **Yanyuan Qiao**[4†], **Qi Wu** [1‡]
[1]Australian Institute for Machine Learning, Adelaide University
[2]Ruyi Dynamics
[3]Fudan University
[4] CREATE Lab, Swiss Federal Institute of Technology Lausanne (EPFL)

[†]Project leader, [‡] Corresponding author

Abstract: Vision-and-Language Navigation (VLN) agents may satisfy conventional success criteria while still failing to establish reliable object-level grounding, because current evaluation protocols mainly reward stopping within a 3-meter radius and largely ignore the agent's final orientation and target visibility. We formalize this limitation as the Last-3-Meter Grounding Gap and introduce three instance-centric metrics to quantify proximity precision, target visibility, and final-view grounding. To mitigate this gap, we propose REALM (Region-to-Entity Alignment for Last-3-Meter Navigation), a plug-and-play, architecture-agnostic refinement module that decouples fine-grained target approaching from long-horizon navigation. REALM uses a visibility-aware stopping strategy to reduce premature termination and improve final viewpoint alignment. We further construct REVERIE-AIM, which provides object-instance-level goals and 180K short-horizon training samples for final-stage target approaching. Evaluations across four diverse VLN backbones show that REALM consistently improves proximity precision and visual grounding success, suggesting its potential applicability.

Keywords: Vision-and-Language Navigation, Object-Centric Navigation

1 Introduction

For embodied agents to follow language instructions in physical environments, reaching the correct room or region is often insufficient: the agent must also obtain a viewpoint from which the referred object can be reliably perceived and grounded. This capability connects high-level language understanding with object-level visual grounding, and provides a necessary perceptual interface for downstream interaction tasks such as inspection or manipulation. The REVERIE task [1] was the first introduced to study this setting: given a natural language instruction containing a referring expression, the agent must navigate through an indoor environment, locate the specified object instance, and predict its bounding box. REVERIE-CE [2] further transfers this task from discrete navigation graphs to continuous environments, bringing it closer to realistic robotic deployment.

However, current evaluation protocols for REVERIE and REVERIE-CE still largely inherit region-level navigation criteria from VLN-CE. In subsequent methods [3, 4], an agent is typically deemed successful if it stops within 3 meters of a pre-annotated endpoint. This endpoint represents a navigable location near the target object, rather than the object instance itself. Moreover, the agent's final heading is not evaluated: it may stop within a 3m radius while facing a wall, a corridor, or the opposite direction from the target. As a result, a trajectory can be counted as successful even when the referred object is not visible from the final viewpoint. This limitation is illustrated in Figure 1: while existing evaluation protocols reward the agent for merely stopping near a pre-annotated

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

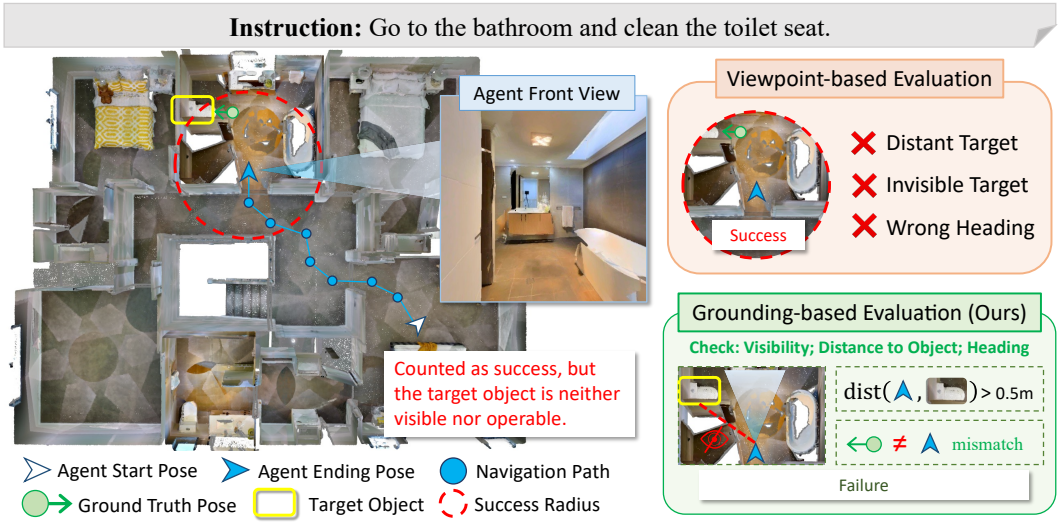


Figure 1: Existing evaluation rewards stopping near a target viewpoint, but object grounding requires a final viewpoint where the referred object is actually visible and localizable.

target region, true object grounding requires the agent to actively adjust its final viewpoint so that the referred object is actually visible and localizable in its camera frame. For tasks that require object-level grounding, such as confirming the object's presence, inspecting its state, or preparing for manipulation, such region-level arrival does not provide a functional endpoint.

While related problems have been studied in other embodied navigation tasks, such as switching strategies near the target [5], optimizing viewpoints via point-cloud scoring [6], and recovering 6-DoF pose from a reference image [7], these methods assume successful target localization or a known goal image, neither of which applies to the VLN setting where the agent must ground a free-form language instruction. We term this systematic gap the **Last-3-Meter Grounding Gap**: the disconnect between coarse-grained region arrival and fine-grained object grounding. Reducing this gap is important for deploying language-conditioned navigation in any real robotic system where the agent must subsequently perceive, manipulate, or interact with the referred object.

To quantify this gap, we propose three complementary object-instance-centric metrics: Object Navigation Success (ONS) for geodesic proximity to the target instance, Grounding Success (GS) for detection accuracy at the final viewpoint, and Oracle Grounding Success (OracleGS) for target visibility in the camera frame. Under these metrics, we find that even ETPNav-FT [8], fine-tuned on the REVERIE-CE dataset, achieves only 6.32% ONS@0.1m and 11.31% OracleGS, in sharp contrast to its 34.67% Success Rate under the conventional 3-meter criterion. This gap shows that strong performance under the conventional Success Rate does not necessarily carry over to instance-level positioning and grounding: methods that rank well on standard VLN metrics can still perform poorly once the target must actually be reached and faced.

To mitigate this gap, we then propose **REALM** (Region-to-Entity Alignment for Last-3-Meter Navigation), a plug-and-play refinement module decoupled from the upstream navigation policy. This decoupled design is motivated by two considerations. Firstly, long-horizon navigation and short-horizon target approach are different problems: the former demands path planning and scene-level reasoning over tens of meters, while the latter requires precise visual alignment and visibility-grounded stopping within a few meters. Coupling them into a single model forces them to share capacity, yielding suboptimal fine-grained behaviour which limits fine-grained performance even when the upstream policy is strong. Secondly, the VLN landscape is highly diverse and rapidly evolving, with methods spanning graph-based, video-based, zero-shot, and training-free paradigms. Tying fine-grained positioning to any specific architecture would require rebuilding the alignment module whenever the upstream model changes, which is neither generalizable nor sustainable. Once any VLN model issues a stop command, REALM takes over using only egocentric RGB and the original language instruction, with no depth sensors, maps, or architectural changes to the upstream

model. We instantiate REALM by LoRA-finetuning UniNaVid [9] with a tailored visibility-aware stopping strategy, which helps reduce premature termination and encourages stopping based on explicit object perception. In addition, to support training and evaluation, we construct **REVERIE-AIM**, which replaces region-level endpoints with object-instance-level goals and provides 180K short-horizon samples for the final approach stage.

Extensive evaluations across four diverse VLN backbones demonstrate that REALM consistently helps reduce the Last-3-Meter Grounding Gap. For instance, it increases ETPNav-ZS's ONS@0.1m from 7.07% to 14.66% (a 107% relative gain) and increases SmartWay's OracleGS from 4.69% to 15.92%. Ablation results suggest that VSP contributes to more precise target approach, with higher ONS at both distance thresholds and improved final-view visibility. Furthermore, a 20-episode evaluation on a Hello Robot Stretch platform provides preliminary evidence of real-world applicability, with REALM improving ONS@0.5m from 10.00% to 35.00%.

To summarise, our contributions are: (1) we identify the **Last-3-Meter Grounding Gap**: the disconnect between reaching the correct region and actually perceiving the target object; (2) we propose **REALM**, a plug-and-play decoupled refinement module with a visibility-aware stopping strategy; (3) we construct the **REVERIE-AIM** dataset and three evaluation metrics (ONS, GS, OracleGS); and (4) we validate REALM across four diverse VLN backbones, demonstrating consistent improvements in fine-grained grounding.

2 Related Work

Vision-and-Language Navigation (VLN) requires an autonomous agent to navigate by following natural language instructions [10, 11, 12, 13]. Early benchmarks primarily operate on discrete environments, where the agent's movement is restricted to predefined navigation graphs [1, 14, 15, 16, 17]. To better align with real-world applications, Krantz et al. [18] lifted these constraints by extending VLN to continuous environments (VLN-CE), enabling agents to move freely without a predefined topological map.

Beyond standard instruction-following VLN, REVERIE [1] introduces a more complex setting that requires agents to navigate via high-level instructions and localise a specific target object. REVERIE-CE [2] further transitions this task into VLN-CE, bringing low-level control and obstacle avoidance challenges into focus. To tackle this, specialised methods like Dynam3D [3] and D3D-VLP [4] leverage dynamic 3D representations on REVERIE-CE, empowering vision-language models with spatial tokens or dynamic chain-of-thought pipelines. Alternatively, advanced paradigms from broader VLN-CE domains, such as graph-based models [8, 19], video-based models [9], or zero-shot navigation approaches [20, 21, 22], can also be adapted to this task. Concurrently, GroundingMate [23] identifies the gap between navigation success and object grounding success in discrete REVERIE, leveraging an MLLM to assist target selection when the agent is uncertain. While effective, its object grounding remains a classification-based paradigm that selects from pre-annotated candidate bounding boxes rather than performing open-world detection, and it operates exclusively in discrete graph environments. However, we observe that both specialised and broader continuous methods inherit a legacy evaluation protocol: success depends solely on stopping within 3 meters of an endpoint, largely ignoring orientation accuracy or object grounding precision. Our work identifies and examines this evaluation gap.

Beyond coarse trajectory following, several works have begun to address fine-grained target approaching. SLING [5] switches navigation strategies upon nearing the target. MSGNav [6] optimises the observation viewpoint via point-cloud visibility scoring after target localisation. AnyImageNav [7] extends the InstanceImageNav success criterion from coarse distance thresholds to precise 6-DoF pose recovery by geometrically registering the goal-image camera orientation. More recently, APRR [24] decouples navigation into an exploration phase and a "last-mile" phase. The latter dynamically refines the robot's pose via active perception to seek a suitable viewpoint for manipulation. However, these methods either assume successful target localisation as a precondition or rely on a known reference image as the navigation goal, neither of which directly transfers to the VLN setting. More importantly, none of them considers the VLN evaluation framework itself:

current benchmarks judge success solely by endpoint proximity while imposing no constraint on final orientation, leaving the *Last-3-Meter Grounding Gap* insufficiently addressed.

3 The Last-3-Meter Grounding Problem

3.1 Preliminaries

For the REVERIE-CE [2] task, at each step t, the navigation policy π_{nav} receives an observation $\mathcal{O}_t$, which may consist of various sensory inputs depending on the system design, including egocentric or panoramic RGB-D views, RGB-only inputs, or other modality-specific features. The agent state at time t is defined as $s_t = (p_t, \theta_t)$, where $p_t \in \mathbb{R}^3$ denotes the 3D position and θ_t denotes the heading direction. Given a high-level natural language instruction $\mathcal{I}$, the agent is required to navigate in an unseen indoor environment $\mathcal{E}$ and reach the goal location. After π_{nav} issues a `stop` command at step T_{nav}, the agent needs to localise the referred target object o^* within its final observation $\mathcal{O}_{T_{\text{nav}}}$ by predicting a bounding box $\hat{b}$: $\hat{b} = \text{Det}(\mathcal{O}_{T_{\text{nav}}}, \mathcal{I})$

3.2 Problem Formulation: The Last-3-Meter Grounding Gap

In standard VLN-CE evaluation, an episode is deemed successful if the agent's stopping position $p_{T_{\text{nav}}}$ lies within 3 meters of a pre-annotated endpoint p_{ep}: $\text{Success Rate (SR)} = [d(p_{T_{\text{nav}}}, p_{\text{ep}}) \leq 3\,\text{m}]$.

However, this paradigm is insufficient for instance-level instruction following [1, 2] because p_{ep} is merely a neighbouring waypoint rather than the physical target. For practical embodied instruction following, the final state $s_{T_{nav}} = (p_{T_{\text{nav}}}, \theta_{T_{\text{nav}}})$ must satisfy two conditions: (i) **Physical proximity** to the actual target instance, and (ii) **Visual groundability** in the egocentric view with sufficient resolution.

These two conditions are purely task-level requirements independent of any particular model design, and together define what we term the **last-3-meter grounding gap**: the discrepancy between reaching the general vicinity of the target (as measured by SR) and achieving a final state from which reliable instance-level grounding is possible.

3.3 Evaluation Metrics

Prior evaluation protocols typically assess navigation and grounding jointly. In contrast, we decouple them into three independent axes to enable fine-grained diagnosis.

Object Navigation Success (ONS). Following ObjectNav [25], we measure navigation precision via the minimum geodesic distance from the agent's stop position to navigable points surrounding the target:

$$\text{ONS@}\tau = \left[\min_{q \in \mathcal{S}_{\text{obj}}} d(p_{T_{\text{nav}}}, q) \leq \tau \right],$$

where $\mathcal{S}_{\text{obj}}$ is the set of navigable points sampled around the target instance and $d(\cdot, \cdot)$ is geodesic distance. Unlike ObjectNav, we require proximity to the specified instance rather than any category match. We report τ=0.1 m (standard ObjectNav) and τ=0.5 m (for downstream positioning tolerance). Note that VLN-CE's SR measures alignment with the annotated endpoint, whereas ONS directly measures proximity to the target object.

Oracle Grounding Success (OracleGS). An episode achieves oracle grounding success if the ground-truth bounding box b^* of the target instance is visible in the agent's current observation $\mathcal{O}_T$ and occupies at least 1% of the camera frame area:

$$\text{OracleGS} = \left[\frac{\text{Area}(b^*)}{\text{Area}(\mathcal{O}_T)} \geq 0.01 \right].$$

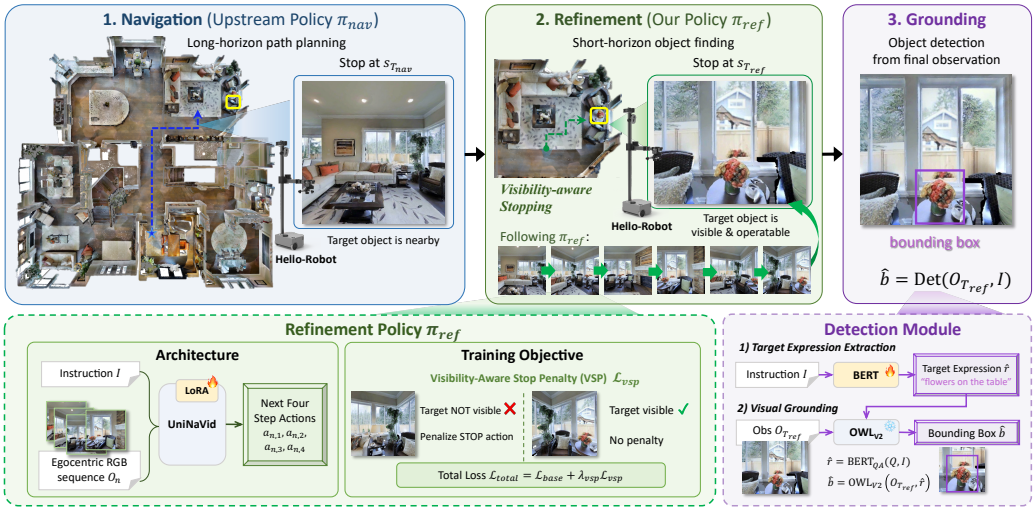


Figure 2: **An overview of the REALM framework.** The pipeline comprises three decoupled stages: **(i) Navigation**: π_{nav} follows instruction $\mathcal{I}$ (e.g., *"Go to the living room and water the flowers on the table."*) and stops at $s_{T_{\text{nav}}}$; **(ii) Refinement**: π_{ref} repositions the agent to $s_{T_{\text{ref}}}$ where the target is proximate and visible; **(iii) Grounding**: a BERT-based extractor yields a target phrase $\hat{r}$, passed to OWLv2 for bounding box prediction. The refinement module is plug-and-play across arbitrary VLN backbones.

Grounding Success (GS). Grounding success is assessed only on episodes that satisfy OracleGS, independent of whether navigation succeeded. Given the predicted bounding box $\hat{b}$ produced by the detection module at the agent's final viewpoint, and the ground-truth bounding box b^*, we define:

$$\text{GS} = \Big[\text{IoU}(\hat{b},\, b^*) \geq 0.5\Big] .$$

By comparing GS with OracleGS, we can distinguish visibility failures caused by suboptimal navigation from failures caused by the detection module.

We evaluate four representative VLN backbones on our proposed REVERIE-AIM to validate this gap. As shown in Table 1, all methods exhibit a performance drop between SR and our metrics. Notably, ETPNav-FT scores 34.67% SR but only 6.32% ONS@0.1m, meaning over 80% of "successes" fail to actually ground the target. This consistent discrepancy across backbones suggests the last-3-meter gap is a systematic issue of the current evaluation protocol.

4 The REALM

We propose **REALM**, a plug-and-play decoupled refinement module to mitigate the above last-3-meter grounding problem. The complete pipeline has three stages (Figure 2): **(i) Navigation**—π_{nav} follows $\mathcal{I}$ and stops at $s_{T_{\text{nav}}}$; **(ii) Refinement**—π_{ref} repositions and reorients the agent to a refined state $s_{T_{\text{ref}}}$ where the target is both proximate and visible; **(iii) Grounding**—a detection module predicts the bounding box over the final observation. To improve efficiency and flexibility, π_{ref} operates fully independently of the upstream navigation policy π_{nav}. This decoupled design aims to reduce performance trade-offs between long-horizon path planning (tens of meters) and short-horizon precision approaching, while supporting plug-and-play compatibility across diverse VLN backbones (e.g., graph-based, video-based, or zero-shot paradigms).

4.1 Refinement Policy

We instantiate the refinement policy π_{ref} with UniNaVid [9], a video-based Vision-Language-Action (VLA) foundation model that maps egocentric video streams and language instructions to multi-step action sequences. We choose UniNaVid for two reasons: its video-level temporal encoding naturally suits the sequential observations arising in short-horizon refinement, and its pre-trained visual-

linguistic representations provide a strong initialisation that may reduce the data requirement for downstream adaptation. At each decision step t, the model consumes the egocentric RGB sequence $\mathcal{O}_t$ together with the instruction $\mathcal{I}$, and predicts the next four-step action sequence $a_{t,1}, a_{t,2}, a_{t,3}, a_{t,4}$ over the low-level action space $\mathcal{A} = \{\texttt{forward}, \texttt{turn_left}, \texttt{turn_right}, \texttt{stop}\}$.

During training, non-action positions (e.g., instruction and visual tokens) are masked with the ignore index -100. Let $\mathcal{M} = \{(b, n) : y_{b,n} \neq -100\}$ denote the set of valid action token positions across a batch of size B with token sequence length N. The baseline imitation learning objective minimizing token-level cross-entropy is:

$$\mathcal{L}_{\text{base}} = \frac{1}{|\mathcal{M}|} \sum_{(b,n)\in\mathcal{M}} \ell_{\text{CE}}(\hat{\mathbf{z}}_{b,n},\, y_{b,n})\,, \tag{1}$$

where $y_{b,n}$ and $\hat{\mathbf{z}}_{b,n}$ represent the target tokens and predicted logits at batch index b and token position n, respectively.

We apply LoRA [26] to all linear layers of the LLM backbone while keeping the vision encoder frozen. However, directly optimizing $\mathcal{L}_{\text{base}}$ suffers from trajectory biases, causing the agent to stop prematurely. We address these failure modes via a visibility-aware stopping strategy.

Visibility-Aware Stop Penalty. During navigation the agent may issue a `stop` action before the target is within view, ending the episode prematurely with no chance of recovery. We penalise this by measuring how strongly the model favours `stop` and applying a penalty only when the target is absent.

Concretely, let $\hat{\mathbf{z}}_b \in \mathbb{R}^{|\mathcal{V}|}$ be the logit vector at the first valid decision step (i.e., $a_{t,1}$ of the four-step sequence $a_{t,1}, a_{t,2}, a_{t,3}, a_{t,4}$ predicted at step t) for sample b, with $\mathcal{V}$ the action vocabulary, and let $v_b \in \{0, 1\}$ indicate target visibility from simulator annotations. We define the *stop margin* to quantify how strongly the model favors stopping over all alternative low-level actions: $m_b = \hat{z}_{b,\,\texttt{stop}} - \max_{k\neq\texttt{stop}} \hat{z}_{b,\,k}$, which is positive exactly when `stop` is the greedy action. To selectively penalize incorrect termination without hindering correct stops, the penalty averages the positive part of this margin over all target-invisible samples:

$$\mathcal{L}_{\text{vsp}} \;=\; \frac{1}{N_{\text{inv}}} \sum_{b=1}^{B} \text{ReLU}(m_b) \;\cdot\; \mathbb{I}[v_b = 0]\,, \tag{2}$$

where $N_{\text{inv}} = \max\big(\sum_b \mathbb{I}[v_b{=}0],\; 1\big)$, $\mathbb{I}[\cdot]$ is the indicator function, and the denominator is clamped to at least 1 for numerical stability. This leaves the agent free to stop once the target is perceived, while suppressing premature termination during exploration.

Total Training Objective. The final loss combines imitation and the stop penalty:

$$\mathcal{L}_{\text{total}} \;=\; \mathcal{L}_{\text{base}} \;+\; \lambda_{\text{vsp}}\, \mathcal{L}_{\text{vsp}}, \tag{3}$$

where λ_{vsp} controls the penalty strength relative to the imitation signal.

4.2 Target Expression Extraction and Visual Grounding

After the refinement policy terminates at $s_{T_{\text{ref}}}$, the agent must localise the target object in its final observation. However, REVERIE-AIM instructions often contain both navigation cues and object descriptions, e.g., *"go to the bathroom and clean the toilet seat"*. Directly using the full instruction as an open-vocabulary detector query may introduce irrelevant spatial or action-related words. We therefore first extract a target phrase from the instruction and then use it for visual grounding.

For target expression extraction, we use GPT-4o-mini to automatically annotate target noun phrases, and fine-tune a BERT-Large [27] extractive QA model on 2,048 curated instruction–phrase pairs. At inference time, the BERT model extracts the target span $\hat{r}$ from the instruction $\mathcal{I}$, $\hat{r} = \text{BERT}_{QA}(\mathcal{I})$. For visual grounding, the extracted $\hat{r}$ is passed to OWLv2 [28] as the text query to predict the final bounding box $\hat{b}$, $\hat{b} = \text{OWL}_{V2}(\mathcal{O}_{T_{\text{ref}}}, \hat{r})$, where $O_{T_{\text{ref}}}$ is the agent's final egocentric observation. We select the bounding box with the highest confidence score as the final prediction $\hat{b}$. This design aims to reduce irrelevant navigation cues from the grounding query.

5 Experiments

5.1 Dataset Construction

The REVERIE-AIM Dataset. To support the proposed instance-level evaluation and provide sufficient supervision for fine-grained target approaching, we construct the **REVERIE-AIM** dataset by extending REVERIE [1]. Specifically, we remap the original discrete trajectories into the Habitat continuous simulator and directly leverage the object instance annotations from ObjectNav [29] for our object-centric goal alignment strategy. This elevates the navigation objective from region-level arrival to precise instance-level alignment. REVERIE-AIM contains 3,691 training episodes across 53 scenes and 1,344 validation episodes (*val-unseen*) across 10 unseen scenes. Crucially, our dataset compresses the average endpoint-to-target geodesic distance from 1.32 m (standard REVERIE-CE) to 0.34 m.

Short-Horizon Fine-Grained Training Split. To provide dense supervision for the final target-approaching stage, we further derive a dedicated short-horizon split from REVERIE-AIM. By applying spatial perturbations and viewpoint augmentations around the target's adjacent nodes, we extract approximately 180,000 fine-grained *instruction–visual observation–action sequence* tuples. This split provides rich low-level action supervision (`turn_left`, `turn_right`, `forward`, `stop`) to reduce the last-three-meter grounding gap. Please refer to the Appendix for detailed discussions.

5.2 Experimental Setup

Upstream VLN Baselines We evaluate our plug-and-play module across four diverse VLN-CE baselines to assess its applicability: (1) ETPNav-ZS [8], a zero-shot topological planning policy, testing refinement under zero exposure to object-referring instructions; (2) ETPNav-FT [8], a fine-tuned variant of ETPNav, evaluating benefits when the upstream policy is already task-calibrated; (3) UniNaVid-ZS [9], a video-based VLA baseline that shares the same base architecture as our module, isolating the gains of our LoRA adaptation; and (4) Smartway [21], a training-free paradigm using large VLMs. These backbones cover zero-shot/fine-tuned, graph/video-based, and training-free navigation paradigms.

Downstream Variants To evaluate the components and optimization of our refinement module, we compare seven downstream variants: (1)w/o Refinement, which directly uses the upstream policy's raw endpoints; (2) Random Walk, which performs random movements at the destination; (3) Heuristic CLIP-Search, which spins 360° and moves toward the direction with the highest CLIP matching score; (4) Det-Guided, a detector-guided refinement that spins 360° with GroundingDINO [30] queried by the same BERT-extracted target phrase used for GS, turns toward the most confident detection, and steps forward while re-detecting and re-centering the target until its box covers 5% of the image or 100 steps are reached; (5) Vanilla UniNaVid, the training-free base architecture executing raw upstream endpoints directly; (6) Ours w/o VSP, our model optimized on our data using standard cross-entropy loss; and (7) Ours (Full), our complete module optimized on our data by the full tailored loss function.

Standard VLN-CE Metrics We report standard VLN-CE metrics (TL, NE, OSR, SR, SPL) [31] for upstream reference. Our instance-level metrics (ONS, GS, OracleGS) are the primary evaluation targets.

5.3 Results in Simulator

As shown in Table 1, our full module (**Ours (Full)**) outperforms most of the alternative move policies across all four backbones. For instance, on ETPNav-FT, it boosts ONS@0.1m from 6.32% (no move) to $\mathbf{17.19}\%$ and GS from 4.46% to $\mathbf{8.48}\%$, indicating its effectiveness in fine-grained object localization. The ablation results suggest that VSP helps refine the agent's final stopping position and orientation, as reflected in the ONS and OracleGS gains. On ETPNav-FT under ONS@0.5m, REALM turns 18.5% of all val-unseen episodes from failure into success, while turning 4.8% from success into failure. Nevertheless, a substantial performance gap remains compared to Human Eval

Table 1: Plug-in fine-grained navigation module performance on REVERIE-AIM (val-unseen). Standard VLN metrics are fixed per backbone and shown in parentheses beside each backbone name. **Bold** = best in group; underline = second best.

Backbone	Move Policy	ONS@0.1m↑	ONS@0.5m↑	GS↑	OracleGS↑
ETPNav-ZS TL = 11.85, NE↓ = 7.26, OSR = 35.86, SPL = 23.39, SR = 28.94					
ETPNav-ZS	— (no move)	7.07	11.98	3.05	6.99
	Random	6.03	12.13	2.46	5.36
	Heuristic	7.51	12.57	4.02	10.19
	Det-Guided	6.32	12.50	5.95	11.01
	Vanilla UniNavid	11.16	17.78	4.54	11.90
	Ours w/o VSP	13.47	20.24	5.28	15.55
	Ours (Full)	**14.66**	**21.13**	**6.10**	**16.52**
ETPNav-FT TL = 13.86, NE↓ = 6.71, OSR = 40.63, SPL = 27.68, SR = 34.67					
ETPNav-FT	— (no move)	6.32	12.20	4.46	11.31
	Random	5.65	12.95	2.75	8.78
	Heuristic	6.77	13.24	5.88	12.05
	Det-Guided	7.14	13.62	6.03	12.50
	Vanilla UniNavid	13.39	19.12	4.91	12.57
	Ours w/o VSP	16.82	25.00	8.04	19.79
	Ours (Full)	**17.19**	**25.89**	**8.48**	**20.24**
UniNavid-ZS TL = 16.07, NE↓ = 8.69, OSR = 34.40, SPL = 17.40, SR = 25.10					
UniNavid-ZS	— (no move)	13.61	18.08	5.65	12.20
	Random	9.75	16.89	2.83	8.78
	Heuristic	12.35	18.15	5.13	10.42
	Det-Guided	12.95	17.86	**7.14**	12.95
	Vanilla UniNavid	10.42	16.07	4.84	11.01
	Ours w/o VSP	12.43	18.38	5.95	14.29
	Ours (Full)	**14.96**	**20.68**	6.85	**16.37**
Smartway TL = 20.00, NE↓ = 8.79, OSR = 25.37, SPL = 4.81, SR = 8.21					
Smartway	— (no move)	7.81	11.83	1.49	4.69
	Random	5.21	10.42	1.34	4.61
	Heuristic	7.22	11.98	2.90	8.04
	Det-Guided	7.22	11.83	4.24	10.04
	Vanilla UniNavid	9.30	14.81	3.50	9.45
	Ours w/o VSP	12.95	17.49	4.69	14.66
	Ours (Full)	**13.99**	**19.72**	**5.65**	**15.92**
Human Eval		50.40	74.71	38.24	80.38

Table 2: Real-world evaluation over 20 episodes on the Hello Robot Stretch platform. All metrics are percentages; bold indicates the better result for each metric.

Method	ONS@0.5m↑	GS↑	OracleGS↑
UniNaVid	10.00	5.00	15.00
UniNaVid + REALM	**35.00**	**20.00**	**25.00**

(e.g., 25.89% vs. 74.71% on ONS@0.5m), highlighting that this benchmark leaves substantial room for future improvement.

5.4 Results in Real-world Environments

To evaluate REALM in physical environments, we compare UniNaVid with and without refinement on a Hello Robot Stretch platform equipped with an Intel RealSense D435if RGB-D camera. Inference runs on a workstation with an NVIDIA RTX 4090 GPU. We evaluate on 20 episodes across a robot lab, a school lobby, and a foyer, covering diverse target categories, including beverage cans. For ONS@0.5m, an episode succeeds if the robot stops within a tape-measured Euclidean distance of 0.5 m from the nearest navigable position associated with the target instance; obstructed paths are counted as failures. GS and OracleGS follow the same definitions as in simulation.

As shown in Table 2, REALM improves all three metrics over the 20 episodes, increasing ONS@0.5m from 10.00% to 35.00%, GS from 5.00% to 20.00%, and OracleGS from 15.00% to

25.00%, corresponding to gains of 25, 15, and 10 percentage points, respectively. These results provide preliminary evidence that refinement improves target proximity, visibility, and grounding in physical environments. However, the limited number of episodes and low absolute success rates warrant further evaluation across a broader range of environments and target objects.

6 Conclusion and Limitations

We identify the Last-3-Meter Grounding Gap in continuous VLN: region-level arrival does not ensure that the referred object is visible and localizable in the final view. REVERIE-AIM and three complementary instance-centric metrics make this gap measurable, while REALM addresses it through decoupled refinement with visibility-aware stopping. Across four VLN backbones, REALM improves target proximity, visibility, and grounding over the upstream stopping states. These results highlight final-view alignment as an explicit objective for object-centric VLN and demonstrate the value of a dedicated refinement stage.

Several limitations remain. Absolute success rates are still substantially below human performance, and refinement can turn some successful upstream endpoints into failures, motivating more reliable viewpoint selection and stopping decisions. GS also depends on the grounding detector: our detector comparison preserves the gains over unrefined baselines but changes absolute scores. These limitations motivate future work on coordinating viewpoint selection, stopping, and visual grounding to support reliable instance-level navigation and its integration with mobile manipulation.

Acknowledgements

We thank Xinyu Yan for sharing helpful ideas during the rebuttal process.

References

[1] Y. Qi, Q. Wu, P. Anderson, X. Wang, W. Y. Wang, C. Shen, and A. v. d. Hengel. Reverie: Remote embodied visual referring expression in real indoor environments. In *CVPR*, pages 9982–9991, 2020.

[2] X. Li, D. Guo, H. Liu, and F. Sun. Reve-ce: Remote embodied visual referring expression in continuous environment. *RAL*, 7(2):1494–1501, 2022.

[3] Z. Wang, S. Lee, and G. H. Lee. Dynam3d: Dynamic layered 3d tokens empower vlm for vision-and-language navigation. In *NeurIPS*, 2025.

[4] Z. Wang, S. Lee, G. Dai, and G. H. Lee. D3d-vlp: Dynamic 3d vision-language-planning model for embodied grounding and navigation. *arXiv preprint arXiv:2512.12622*, 2025.

[5] J. Wasserman, K. Yadav, G. Chowdhary, A. Gupta, and U. Jain. Last-mile embodied visual navigation. In *CoRL*, pages 666–678. PMLR, 2023.

[6] X. Huang, S. Zhao, Y. Wang, X. Lu, W. Zhang, R. Qu, W. Li, Y. Wang, and C. Wen. Msgnav: Unleashing the power of multi-modal 3d scene graph for zero-shot embodied navigation. *arXiv preprint arXiv:2511.10376*, 2025.

[7] Y. Deng, S. Yuan, and Y. Fang. Anyimagenav: Any-view geometry for precise last-meter image-goal navigation. *arXiv preprint arXiv:2604.05351*, 2026.

[8] D. An, H. Wang, W. Wang, Z. Wang, Y. Huang, K. He, and L. Wang. Etpnav: Evolving topological planning for vision-language navigation in continuous environments. *TPAMI*, 2024.

[9] J. Zhang, K. Wang, S. Wang, M. Li, H. Liu, S. Wei, Z. Wang, Z. Zhang, and H. Wang. Uninavid: A video-based vision-language-action model for unifying embodied navigation tasks. *arXiv preprint arXiv:2412.06224*, 2024.

[10] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, and A. Van Den Hengel. Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments. In *CVPR*, pages 3674–3683, 2018.

[11] Y. Zhang, Z. Ma, J. Li, Y. Qiao, Z. Wang, J. Chai, Q. Wu, M. Bansal, and P. Kordjamshidi. Vision-and-language navigation today and tomorrow: A survey in the era of foundation models. *arXiv preprint arXiv:2407.07035*, 2024.

[12] X. Su, R. Chen, B. Liu, J. Ma, Z. Di, R. Krishna, and J. Froehlich. Capnav: Benchmarking vision language models on capability-conditioned indoor navigation. *arXiv preprint arXiv:2602.18424*, 2026.

[13] S. Wang, Y. Wang, G. Lian, Y. Wang, M. Chen, K. Wang, B. Zhang, Z. Su, Y. Zhou, W. Li, et al. Progress-think: Semantic progress reasoning for vision-language navigation. In *CVPR*, 2026.

[14] A. Ku, P. Anderson, R. Patel, E. Ie, and J. Baldridge. Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding. In *EMNLP*, pages 4392–4412, 2020.

[15] J. Thomason, M. Murray, M. Cakmak, and L. Zettlemoyer. Vision-and-dialog navigation. In *CoRL*, pages 394–406, 2020.

[16] Y. Qiao, Y. Qi, Y. Hong, Z. Yu, P. Wang, and Q. Wu. Hop: History-and-order aware pre-training for vision-and-language navigation. In *CVPR*, pages 15418–15427, 2022.

[17] J. Zhang, J. Fan, J. Peng, et al. Curriculum learning for vision-and-language navigation. *NeurIPS*, 34:13328–13339, 2021.

[18] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee. Beyond the nav-graph: Vision-and-language navigation in continuous environments. In *ECCV*, pages 104–120. Springer, 2020.

[19] S. Zhang, Y. Qiao, Q. Wang, Z. Yan, Q. Wu, Z. Wei, and J. Liu. Cosmo: Combination of selective memorization for low-cost vision-and-language navigation. In *ICCV*, pages 5511–5522, 2025.

[20] Y. Long, W. Cai, H. Wang, G. Zhan, and H. Dong. Instructnav: Zero-shot system for generic instruction navigation in unexplored environment. In *CoRL*, pages 2049–2060. PMLR, 2024.

[21] X. Shi, Z. Li, W. Lyu, J. Xia, F. Dayoub, Y. Qiao, and Q. Wu. Smartway: Enhanced waypoint prediction and backtracking for zero-shot vision-and-language navigation. In *IROS*, 2025.

[22] K. Chen, D. An, Y. Huang, R. Xu, Y. Su, Y. Ling, I. Reid, and L. Wang. Constraint-aware zero-shot vision-language navigation in continuous environments. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

[23] Q. Liu, S. Zhang, Y. Qiao, J. Zhu, X. Li, L. Guo, Q. Wang, X. He, Q. Wu, and J. Liu. Groundingmate: Aiding object grounding for goal-oriented vision-and-language navigation. In *WACV*, pages 1775–1784. IEEE, 2025.

[24] L. Qin, M. Wang, P. Li, W. Zhou, and H. Li. Active perception meets rule-guided rl: A two-phase approach for precise object navigation in complex environments. In *ICCV*, pages 7603–7612, 2025.

[25] M. Khanna, Y. Mao, H. Jiang, S. Haresh, B. Shacklett, D. Batra, A. Clegg, E. Undersander, A. X. Chang, and M. Savva. Habitat synthetic scenes dataset (hssd-200): An analysis of 3d scene scale and realism tradeoffs for objectgoal navigation. In *CVPR*, pages 16384–16393, 2024.

[26] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. In *ICLR*. OpenReview.net, 2022.

[27] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[28] M. Minderer, A. Gritsenko, and N. Houlsby. Scaling open-vocabulary object detection. *NeurIPS*, 36:72983–73007, 2023.

[29] D. Batra, A. Gokaslan, A. Kembhavi, O. Maksymets, R. Mottaghi, M. Savva, A. Toshev, and E. Wijmans. Objectnav revisited: On evaluation of embodied agents navigating to objects. *arXiv preprint arXiv:2006.13171*, 2020.

[30] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In *European conference on computer vision*, pages 38–55. Springer, 2024.

[31] Y. Hong, Z. Wang, Q. Wu, and S. Gould. Bridging the gap between learning in discrete and continuous environments for vision-and-language navigation. In *CVPR*, pages 15439–15449, 2022.

[32] S. Chen, T. Chabal, I. Laptev, and C. Schmid. Object goal navigation with recursive implicit maps. In *IROS*, pages 7089–7096. IEEE, 2023.

[33] K. Yadav, S. K. Ramakrishnan, J. Turner, A. Gokaslan, O. Maksymets, R. Jain, R. Ramrakhya, A. X. Chang, A. Clegg, M. Savva, E. Undersander, D. S. Chaplot, and D. Batra. Habitat challenge 2022. https://aihabitat.org/challenge/2022/, 2022.

[34] K. Yadav, R. Ramrakhya, S. K. Ramakrishnan, T. Gervet, J. Turner, A. Gokaslan, N. Maestre, A. X. Chang, D. Batra, M. Savva, et al. Habitat-matterport 3d semantics dataset. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4927–4936, 2023.

Appendix

A Overview of the Supplementary Material

This supplementary document provides additional technical details, dataset analyses, and extended experimental configurations to complement the main manuscript. The contents are organized as follows:

- **The REVERIE-AIM Dataset (Section B):** We elaborate on the progressive stages of constructing our instance-level dataset, including continuous trajectory reconstruction, object-centric goal alignment, comprehensive dataset statistics, and the generation pipeline for short-horizon dense supervision.
- **Implementation Details (Section C.1):** We provide the hyperparameters, training configurations, and low-rank adaptation (LoRA) [26] settings utilized for optimizing our navigation policy backbones.
- **Real-Robot Deployment Setup (Section C.2):** We describe the hardware platform and deployment configuration used for real-world validation, including the Hello Robot Stretch 3 platform and the evaluated indoor scenes.
- **Computational and Execution Costs (Section C.3):** We report inference latency and GPU memory usage for the refinement and grounding modules, as well as the additional action steps and travel distance introduced by refinement.
- **Target Expression Extraction and Visual Grounding (Section C.4):** We detail our two-stage target localization framework, showcasing the prompt template used for automated LLM annotation and the training formulations of our BERT-QA [27] span extractor.
- **Additional Experimental Results (Section D):** We report ONS at a 3 m threshold, evaluate the contribution of target phrase extraction, and analyze sensitivity to the grounding detector and the Visibility-aware Stop Penalty (VSP) weight λ_{vsp}.
- **Broader Impact (Section E):** We discuss collision risks and LiDAR-based obstacle avoidance, as well as privacy measures including local inference, image deletion after processing, and the potential benefits of future onboard inference.

B The REVERIE-AIM Dataset

To support the instance-level evaluation metrics introduced in the main content and provide sufficient supervision for both long-horizon navigation and fine-grained target approaching, we construct the REVERIE-AIM dataset by extending the original REVERIE [1] through three progressive stages. First, we remap the discrete navigation graph into the Habitat continuous simulator to obtain physically plausible trajectories (Section B.1). Second, we replace the original region-level goal annotations with instance-level object-centric endpoints aligned via ObjectNav sampling (Section B.2). Third, we summarize the dataset statistics and compare against existing benchmarks (Section B.3). We then extract a dedicated short-horizon training set from REVERIE-AIM to provide dense supervision for the final-meters approaching phase (Section B.4).

B.1 Continuous Trajectory Reconstruction

The original REVERIE dataset [1] is constructed upon a discrete navigation graph, where the agent's movement is restricted to transitions between predefined nodes. Such a mechanism does not fully capture the dynamic complexity of continuous motion and the fine-grained requirements of obstacle avoidance. Inspired by the trajectory transfer strategies adopted in REVERIE-CE [2] and Dynam3D [3], we systematically remap the discrete node-based trajectories in REVERIE into the Habitat continuous simulation environment.

Table 3: Comparison with existing embodied navigation datasets. Our dataset is the first to combine all four desirable properties.

Dataset	Continuous Space	Language Goal	Instance-level Target	Object Proximity Endpoint
Vision-and-Language Navigation (Discrete)				
R2R [10]	×	✓	×	×
RxR [14]	×	✓	×	×
REVERIE [1]	×	✓	✓	×
Vision-and-Language Navigation (Continuous)				
R2R-CE [18]	✓	✓	×	×
RxR-CE [14, 18]	✓	✓	×	×
REVERIE-CE [2]	✓	✓	✓	×
Object Navigation				
MP3D ObjNav [29]	✓	×	×	✓
HM3D ObjNav [33]	✓	×	×	✓
HM3D ImageNav [34]	✓	×	✓	✓
Ours	✓	✓	✓	✓

Specifically, through trajectory mapping and interpolation, the agent's action space is transformed from discrete node transitions into continuous linear movements and arbitrary-angle rotations. This process aims to ensure that the generated trajectories exhibit both motion smoothness and physical plausibility within a continuous action space.

B.2 Object-Centric Goal Alignment

The REVERIE dataset defines targets as nearby navigable waypoints rather than the physical boundaries of the instances themselves. Consequently, training on such loosely annotated goals limits agents to region-level arrival instead of precise instance-level localization.

To address this limitation, we integrate the annotation protocol from the Habitat Object Goal Navigation (ObjectNav) task [32] and propose an object-instance-centric annotation fusion strategy. Capitalizing on the shared Matterport3D scene hierarchy between REVERIE and ObjectNav, we establish a deterministic mapping of instance identifiers across both benchmarks. As a result, the sampling locations generated by ObjectNav for target instances can be transferred into our dataset framework without information loss.

Specifically, we replace the original ambiguous nearby-goal annotations with precise sampling points located within the geometric boundary or visually observable region of the target object. This modification elevates the navigation objective from mere **position reachability** to **instance-level precise alignment**.

After completing continuous trajectory reconstruction and object-centric goal alignment, the REVERIE-AIM dataset possesses the fundamental structure required for long-horizon navigation training.

B.3 Dataset Statistics

The resulting REVERIE-AIM dataset contains 3,691 training trajectories spanning 53 scenes and covering 90 target object categories, as well as 1,344 validation trajectories (*val-unseen*) across 10 unseen scenes with 48 object categories. To contextualize the scalability and structural properties of REVERIE-AIM within the robot learning community, we present a feature-matrix comparison against existing embodied navigation benchmarks in Table 3.

Our object-centric goal alignment helps reduce the region-to-object grounding gap. While standard REVERIE-CE endpoints terminate at a loose, region-level distance, REVERIE-AIM reduces the

valid average minimum geodesic distance to the target from 1.32 m to just 0.34 m. This finer-grained spatial resolution enables more precise evaluation of both navigation and instance-level grounding.

B.4 Short-Horizon Training Data Generation

We further construct a dedicated fine-grained navigation dataset based on REVERIE-AIM. The primary objective of this stage is to generate a large number of short-horizon video clips containing *instruction–visual observation–action sequence* tuples, thereby enhancing the agent's decision-making capability during the target interaction stage.

Task-specific Initial Sampling To simulate the visual state of an agent approaching the final stage of long-horizon navigation, we select the second-to-last node of the reference path as the anchor point for initialization. To further improve the model's tolerance to localization errors and enhance generalization, we augment the initial states through the following strategies:

Spatial Perturbation. We perturb the anchor position using a truncated Gaussian distribution ($\sigma = 1.5$, truncated within $[-5, 5]$ meter) to generate spatially diverse starting states around the target object.

Viewpoint Augmentation. We randomly sample the initial heading angle within $[-\pi, \pi]$ to simulate diverse target-approaching viewpoints.

Clip and Action Sequence Extraction For each valid sampled starting point, we compute the shortest path to the target object's surrounding sampling location, determined by the object-centric endpoint described in Section B.2. We then extract the corresponding visual observation sequences (video clips) together with their continuous action labels. The action labels are a set of low-level actions: {`forward`, `turn_left`, `turn_right`, `stop`}. In total, the resulting dataset contains approximately 180,000 fine-grained navigation samples from the training split. These samples provide dense supervision for learning precise short-horizon navigation behaviors, particularly during the final target-approaching stage.



Figure 3: The Hello Robot Stretch 3 platform used in our real-world experiments.

C Additional Details

C.1 Implementation Details

All models are fine-tuned from `UniNaVid-7B` [9] using Low-Rank Adaptation (LoRA) [26] with rank $r = 128$, scaling factor $\alpha_{\text{LoRA}} = 256$, and dropout 0.05. Training is conducted on a single NVIDIA A100 GPU. We train for 2 epochs with a per-device batch size of 8 and gradient accumulation steps of 8, yielding an effective batch size of 64. The learning rate is set to 1×10^{-5} with a warmup ratio of 0.03 and a constant-with-warmup schedule. Input videos are sampled at 1 FPS. λ_{vsp} is set as 1.

C.2 Real-Robot Deployment Setup

As shown in Figure 3, all real-world experiments are conducted on the Hello Robot platform, Stretch 3, which is equipped with an Intel RealSense D435if RGB-D camera. The robot receives high-level navigation instructions and executes low-level actions {`forward`, `turn_left`, `turn_right`, `stop`}, consistent with the action space defined in the simulation. All inference is executed on a workstation with the NVIDIA RTX 4090 GPU during operation. We evaluate our system across various real-world indoor scenes covering diverse room configurations, including a robot lab, a school lobby, and a foyer, with two representative scenes illustrated in Figure 4 and Figure 5.

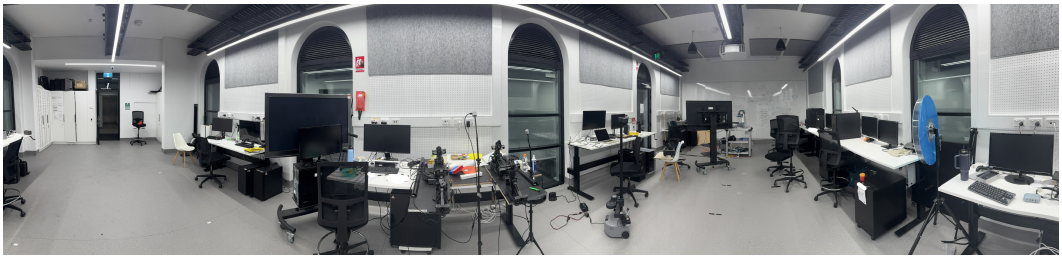


Figure 4: Real-world robot experiments in the lab.



Figure 5: Real-world robot experiments in the lobby.

C.3 Computational and Execution Costs

On a workstation equipped with NVIDIA RTX 4090 GPU and AMD Ryzen 9 7900X CPU, REALM requires 147.69 ms per inference call. Following UniNaVid, we execute the first two predicted actions after each call, yielding an amortized inference time of 73.85 ms per action. The grounding module requires 171.4 ms per image. The refinement and grounding modules use 15.6 GB and 3.1 GB of GPU memory, respectively, and are invoked only after the upstream navigation policy terminates.

The refinement stage takes an average of 56 action steps per episode, including turns, and adds 8.05 m of travel on average. This travel distance is 1.13 times the 7.11 m distance from the upstream stopping position to the target.

C.4 Target Expression Extraction and Visual Grounding

After the refinement policy terminates at $s_{T_{\text{ref}}}$, the agent proceeds to localise the target object in its final observation. However, the instructions in REVERIE-AIM contain rich directional semantics (e.g., *"go to the second floor and turn left"*) interleaved with object descriptions. Passing the full instruction directly to an open-vocabulary detector introduces substantial noise, as navigational cues are irrelevant to visual grounding. We therefore decompose target localisation into two stages: *target expression extraction*, which distils the instruction into a concise noun phrase, and *visual grounding*, which detects the corresponding object in the final observation.

Manually annotating target spans for all training instructions is labor-intensive. Instead, we leverage a large language model (GPT-4o-mini) to automatically extract concise target noun phrases from each instruction in a zero-shot manner. The extracted phrases are then aligned back to the original instruction tokens to obtain start and end positions, producing span-level supervision for training. The prompt template used for extraction is shown below:

```
Given the following navigation instruction, extract
the target object the person needs to interact with.
Return a short noun phrase (ideally 2-6 words) that
can be used as a visual grounding query.  Include only
```

```
the most distinguishing attributes.  Return ONLY the
phrase, nothing else.
Instruction:  {instruction}
Target Object:
```

In total, we collect 3,468 instruction-target pairs through automatic extraction. We first split data into training, validation, and test sets using an approximate 70/15/15 split, resulting in 2,417 initial training pairs. After manual data cleaning, we keep 2048 high-quality pairs for training.

We frame target extraction as extractive question answering. Given a navigation instruction I as the context and a fixed query *"What is the target object to manipulate?"*, a BERT-Large [27] model predicts the start and end token positions that delimit the target expression within I. We initialize from `bert-large-uncased-whole-word-masking-finetuned-squad` and fine-tune on the LLM-annotated training split with the standard span extraction loss:

$$\mathcal{L}_{\text{text}} = -\log p_{s^*}^{\text{start}} - \log p_{e^*}^{\text{end}}, \tag{4}$$

where (s^*, e^*) denotes the ground-truth span boundaries. At inference, the model yields a concise noun phrase $\hat{r}$ (e.g., *"white cup on the counter"*) without requiring any external API.

The extracted phrase $\hat{r}$ is passed as the text query to OWLv2 [28], an open-vocabulary detector, to predict the target bounding box $\hat{b}$ in the agent's final observation $O_{T_{\text{ref}}}$:

$$\hat{r} = \text{BERT}_{\text{QA}}(Q,\, I), \qquad \hat{b} = \text{OWLv2}(O_{T_{\text{ref}}},\, \hat{r}). \tag{5}$$

By replacing the full instruction with a short, semantics-clean phrase, we aim to reduce ambiguity in the grounding stage.

D Additional Results

D.1 Evaluation at a 3 m Threshold

REALM improves ONS@3m by 0.8–5.8 percentage points over the upstream stopping positions across all four backbones. This uses the same 3 m threshold as VLN-CE SR, but measures proximity to the target instance rather than the annotated region-level navigation endpoint.

D.2 Ablation Study on Target Expression Extraction

We evaluate on the target-visible subset of the *val-unseen* episodes. This subset is derived from the best-performing configuration, ETPNav-FT with our method, which yields the highest proportion of target-visible final observations. Specifically, the target object is visible in 20.24% of the 1,344 validation episodes, resulting in a total of 272 episodes as reported in the main paper. Table 4 evaluates the contribution of target expression extraction on visual grounding performance, measured by the success rate over 272 validation episodes. When the full instruction is passed directly to OWLv2 [28] as the detection query (w/o Extraction), the agent achieves a success rate of 30.88%, as navigational cues and action-oriented language (e.g., *"go to the second floor and turn left"*) introduce substantial noise into the grounding query. By extracting a concise target noun phrase via BERT-QA and passing it to OWLv2, our method raises the success rate to 41.91%, a gain of 11.03 percentage points. This suggests that decomposing localisation into extraction followed by grounding helps reduce irrelevant instruction semantics and allows the detector to focus on visually discriminative object attributes.

Table 4: Ablation study on target expression extraction.

Method	**Success**	**Total**	**Success Rate (%)**
w/o Extraction	84	272	30.88
Ours	114	272	41.91

Table 5: GS (%) on REVERIE-AIM (val-unseen) with two grounding detectors, evaluated on identical final observations and target phrases. OWLv2 is the detector used in Table 1. Δ = Ours (Full) − no move.

	OWLv2 (main paper)			GroundingDINO-B		
Backbone	no move	Ours (Full)	Δ	no move	Ours (Full)	Δ
ETPNav-ZS	3.05	6.10	+3.05	3.27	7.51	+4.24
ETPNav-FT	4.46	8.48	+4.02	4.39	10.42	+6.03
UniNavid-ZS	5.65	6.85	+1.20	6.77	7.89	+1.12
Smartway	1.49	5.65	+4.16	2.16	7.59	+5.43
Human Eval		38.24			46.87	

Table 6: Ablation study on the VSP penalty weight λ_{vsp} across four backbones on REVERIE-AIM (val-unseen). Bold = best per column within each backbone.

Backbone	λ_{vsp}	ONS@0.1m ↑	ONS@0.5m ↑	GS ↑	OracleGS ↑
ETPNav-FT	w/o VSP	16.82	25.00	8.04	19.79
	0.1	15.92	22.77	8.33	18.53
	0.5	**17.71**	23.51	**8.56**	19.27
	1 (ours)	17.19	**25.89**	8.48	20.24
	2	17.34	24.26	8.41	**20.54**
	5	16.37	22.69	7.66	19.12
ETPNav-ZS	w/o VSP	13.47	20.24	5.28	15.55
	0.1	14.43	20.31	6.47	17.04
	0.5	13.32	20.31	6.92	16.29
	1 (ours)	14.66	21.13	6.10	16.52
	2	**14.88**	**21.73**	6.70	**18.23**
	5	14.21	19.64	**7.44**	16.59
UniNaVid-ZS	w/o VSP	12.43	18.38	5.95	14.29
	0.1	14.36	20.91	6.70	16.07
	0.5	14.29	19.87	6.99	17.04
	1 (ours)	14.96	20.68	6.85	16.37
	2	**16.22**	**21.50**	**7.59**	**17.11**
	5	14.14	19.27	6.85	16.96
Smartway	w/o VSP	12.95	17.49	4.69	14.66
	0.1	11.83	18.15	5.28	14.66
	0.5	12.72	18.01	5.73	15.85
	1 (ours)	**13.99**	**19.72**	5.65	15.92
	2	12.13	17.41	6.32	14.58
	5	13.69	18.97	**6.40**	**16.07**

D.3 Sensitivity of GS to the Grounding Detector

GS in the main paper uses OWLv2 [28] as the open-vocabulary detector. To test whether our conclusions depend on this choice, we re-evaluate GS on exactly the same final observations and with the same BERT-extracted target phrases, replacing OWLv2 with GroundingDINO (Swin-B) [30]. All other settings follow the main paper.

GroundingDINO improves GS in 26 of 28 backbone–policy combinations (1.2 percentage points on average). With either OWLv2 or GroundingDINO, REALM achieves higher GS than the no-move baseline on all four backbones (Table 5). Changing between these detectors affects absolute GS values but preserves the benefit of refinement.

D.4 Sensitivity to λ_{vsp}

Table 6 reports the effect of the VSP penalty weight λ_{vsp} across four backbones on the REVERIE-AIM val-unseen set. While $\lambda = 2$ achieves higher ONS scores on ETPNav-ZS and UniNaVid-ZS, it degrades performance on Smartway, where ONS@0.1m drops from 13.99 to 12.13 compared to $\lambda = 1$. In the reported evaluations, $\lambda = 1$ improves ONS at both distance thresholds and OracleGS over the w/o VSP baseline on all four backbones, supporting its use as our default for target approach and viewpoint refinement. We therefore adopt $\lambda = 1$ in all main experiments. The best weight varies across backbones and metrics, so a larger penalty does not necessarily yield better navigation performance.

E Broader Impact

Navigation around people introduces collision risks, motivating the integration of navigation policies with the robot's onboard LiDAR for obstacle avoidance. Our current system relies on the robot's LiDAR-based obstacle avoidance, which operates independently of REALM. Privacy is particularly important in home environments, where camera observations may contain personal information. In our setup, all inference runs on a local server, which returns action commands to the robot, and each image frame is deleted after processing. Future deployment with inference performed entirely onboard could further reduce privacy risks by avoiding image transmission to an external server.